

Universidade Federal de Sergipe
Campus de São Cristóvão
Departamento de Computação e Sistemas de Informação

COMP0472 - Sistemas Operacionais
Prof. Glauco Estácio Gonçalves

RELATÓRIO DE AVALIAÇÃO 1 (AV1)
Trilha C: RAG sobre PDF com Modelos de Linguagem Locais

Álex Felipe Santos Melo
Matrícula: 202200025395

São Cristóvão - SE
15 de setembro de 2026

1-Objetivo

Este relatório documenta os experimentos realizados na Avaliação 1 (AV1) da disciplina COMP0472 - Sistemas Operacionais, Trilha C, cujo objetivo é estudar o comportamento de sistemas operacionais durante a execução de aplicações de inteligência artificial local. O trabalho abrange a configuração de um pipeline RAG (Retrieval-Augmented Generation) sobre PDF, a análise de processos e chamadas de sistema (syscalls) durante a inferência, e a medição de desempenho sob diferentes configurações de hardware e software.

Especificamente, buscou-se: (1) compreender como o SO gerencia CPU, memória e sincronização de threads durante inferência de LLM em CPU; (2) quantificar o impacto de variáveis como tamanho de query, concorrência, RAG ativo, modelo menor e restrição de contexto na latência de resposta e uso de RAM; (3) identificar os gargalos do sistema e as alavancas mais efetivas para reduzi-los.

2-Identificação do Discente

Nome: Álex Felipe Santos Melo

Matrícula: 202200025395

Curso: Engenharia de Computação

Disciplina: COMP0472 - Sistemas Operacionais

Professor: Glauco Estácio Gonçalves

Semestre: 2026.2

Data de entrega: 15 de setembro de 2026

3-Materiais e Métodos

Hardware

Todos os experimentos foram executados em notebook com CPU AMD Ryzen 7 7735HS (3,20 GHz), GPU integrada AMD Radeon Graphics (2 GB) e 16 GB de RAM instalada, dos quais 7,8 GB estavam disponíveis para o WSL2. O sistema operacional hospedeiro é Windows 11, com WSL2 rodando Ubuntu 22.04 como ambiente de execução.

Software

O stack de software utilizado foi: Ollama 0.6.2 como servidor de inferência; modelo principal DeepSeek-R1-Distill-Qwen-7B em formato GGUF Q4_K_M (4,7 GB em disco); modelo de embeddings nomic-embed-text para o pipeline RAG; modelo auxiliar deepseek-r1:1.5b (~1 GB) para o experimento C3-A. A aplicação RAG foi ollama_pdf_rag v3.0.0 (commit f17f9c9), baseada em Streamlit, ChromaDB e LangChain.

Métodologia de Medição

As medições de TTFT (Time to First Token, em segundos) e tempo total (em milissegundos) foram obtidas via API REST do Ollama. O uso de RAM foi medido com ps/htop em cada experimento. Para garantir consistência com os gráficos gerados, os cálculos de média utilizam as últimas 2 repetições (tail(2)) de cada sub-configuração.

4-Descrição da Aplicação RAG - Parte A

Identificação

Repositório: dusty-nv/ollama_pdf_rag (GitHub)

Versão: v3.0.0 (commit f17f9c9)

Licença: MIT

Descrição: Aplicação de RAG sobre documentos PDF utilizando modelos de linguagem locais via Ollama. Permite carregar um PDF, indexar seu conteúdo em banco de vetores ChromaDB e responder perguntas sobre o documento usando o modelo configurado.

Arquitetura

A aplicação é composta por: interface web Streamlit (porta 8501); pipeline LangChain para orquestração de RAG; ChromaDB como banco de vetores local; Ollama como servidor de inferência (porta 11434). O fluxo de dados é: PDF -> PyPDF2 -> chunks de texto -> embeddings (nomic-embed-text) -> ChromaDB. Na consulta: query -> embedding -> busca por similaridade (top-k chunks) -> prompt aumentado -> LLM -> resposta.

Configuração Utilizada

O PDF indexado foi o material de estudo da disciplina. O modelo de embeddings foi nomic-embed-text; o modelo de geração foi deepseek-r1:7b. O chunk size padrão foi 1000 tokens com 200 de sobreposição. O número de chunks recuperados por consulta (k) foi 3.

5-Arquitetura de Processos e Syscalls - Parte B

Hierarquia de Processos

Durante a execução do experimento, o sistema apresentou a seguinte hierarquia de processos relevantes:

ollama serve (PID 157, PPID 1/systemd): processo servidor principal, 12 threads, responsável por receber requisições HTTP na porta 11434 e despachar para os workers llama-server.

llama-server deepseek-r1:7b (PID 1059, 11 threads): processo de inferência para o modelo 7B, responde por ~97% do uso de CPU durante a geração de tokens. Utiliza BLAS para multiplicações matriciais paralelizadas.

llama-server nomic-embed-text (PID 1029, 11 threads): processo de embeddings para o pipeline RAG. Ativo durante C2-B; ocioso nas demais configurações.

Streamlit (PIDs 846/847, 21 threads): interface web da aplicação RAG. Presente em memória mesmo quando não há interação ativa.

Perfil de Syscalls - strace no ollama serve

A análise via strace do processo ollama serve durante a inferência revelou a seguinte distribuição de chamadas de sistema:

Tabela 5.1: Distribuição de syscalls no processo ollama serve durante inferência

Syscall	Chamadas	% do total	Função no contexto de LLM em CPU
futex	~2.500.000	76,59%	Sincronização entre as 11 threads do llama-server por barreira de camada transformer
nanosleep	~515.000	15,81%	Polling com espera passiva para evitar busy-waiting em filas internas do Ollama

Syscall	Chamadas	% do total	Funcao no contexto de LLM em CPU
sched_yield	~75.000	2,31%	Cede CPU voluntariamente entre operações de matrix multiply
epoll_pwait	~55.000	1,69%	Multiplexação de I/O HTTP para detectar novas requisições no loopback
sched_getaffinity	~6.500	0,20%	Consulta de afinidade de CPU para distribuição de threads BLAS

Fonte: Autores.

A dominância do futex (76,59%) é característica de aplicações multithread CPU-bound: cada uma das centenas de camadas do transformer exige uma barreira de sincronização entre as 11 threads do llama-server. Como o futex opera majoritariamente em userspace quando não há contenção, o overhead de kernel é mínimo mesmo com milhões de chamadas.

O nanosleep (15,81%) reflete o mecanismo de polling passivo do servidor Ollama enquanto aguarda a conclusão da inferência antes de enviar a resposta HTTP. Isso evita busy-waiting (que consumiria 100% de CPU em espera ativa) e permite que o SO utilize o tempo para outros processos.

6-Configuração do Ambiente de Experimentos - Parte C

Todos os experimentos da Parte C foram conduzidos na mesma sessão, em 15/09/2026, sem reinicialização do sistema entre as configurações. Isso é intencional: permite observar efeitos cumulativos de carregamento de modelos e crescimento de KV-cache ao longo da sessão.

Ambiente: WSL2 (Ubuntu 22.04), CPU Intel x64, 7,8 GB de RAM, sem GPU.

Servidor: Ollama 0.6.2, porta 11434, modo CPU (GGUF Q4_K_M).

Modelo principal: DeepSeek-R1-Distill-Qwen-7B, 4,7 GB em GGUF Q4_K_M, contexto padrão de 8192 tokens.

Medição: TTFT via campo "eval_duration" da API /api/generate; RAM via "ps -o rss" somado para todos os processos ollama/llama-server.

7-Experimentos Realizados

C1-A: Padrão - Query Curta

Query: "O que é escalonamento de processos?" (definição em 2 frases). Configuração padrão do Ollama, sem RAG, sem concorrência. Mede a latência base do modelo 7B para respostas curtas.

C1-B: Padrão - Query Longa

Query: "Explique detalhadamente o escalonamento de processos no Linux, incluindo CFS, prioridades, contextos de troca e exemplos práticos." Mesma configuração de C1-A, mas com query que demanda resposta longa. Mede o impacto do número de tokens gerados na latência.

C2-A: Concorrência - 2 Requisições Simultâneas

Duas requisições idênticas enviadas simultaneamente ao Ollama via curl em background. Mede o comportamento do servidor sob carga concorrente e o overhead de serialização.

C2-B: RAG Pipeline Ativo

Mesma query curta de C1-A, mas com a aplicação Streamlit/ChromaDB ativa e consumindo memória. A inferência é direta (sem passar pelo pipeline RAG), isolando o efeito da pressão de memória causada pelo stack RAG.

C3-A: Modelo Menor - DeepSeek-R1:1.5b

Mesma query curta, mas usando o modelo deepseek-r1:1.5b (~1 GB, GGUF Q4_K_M). O modelo 7B permaneceu carregado simultaneamente. Med e o ganho de latência ao reduzir o tamanho do modelo.

C3-B: Contexto Restrito - num_ctx=2048

Modelo 7B com contexto reduzido de 8192 para 2048 tokens. A redução de contexto diminui a KV-cache máxima alocada em 4x. Med e o impacto da redução de contexto na latência e no uso de RAM.

8-Resultados

Tabela Consolidada

A tabela a seguir apresenta os dados das últimas 2 repetições por sub-configuração, utilizados para cálculo das médias.

Tabela 8.1: Dados brutos das 12 execuções válidas (últimas 2 por sub-configuração)

Config	Variante	Rep	TTFT (s)	Total (ms)	RAM (MB)	Threads	Obs
C1-A	padrão/curta	1	82,90	83.244	5.884	15	
C1-A	padrão/curta	2	36,57	36.972	5.912	15	
C1-B	padrão/longa	1	526,49	528.031	6.166	15	
C1-B	padrão/longa	2	573,77	575.639	6.290	15	
C2-A	concorrente	1	139,07	230.013	5.315	15	2 req simultâneas
C2-A	concorrente	2	84,42	117.180	5.346	15	2 req simultâneas
C2-B	rag-ativo/curta	1	81,68	82.184	6.480	15	RAG pipeline ativo
C2-B	rag-ativo/curta	2	57,54	57.998	6.505	15	RAG pipeline ativo
C3-A	quant-1.5b	1	23,37	55.324	6.602	26	modelo menor
C3-A	quant-1.5b	2	31,35	31.542	6.616	26	modelo menor
C3-B	ctx-2048	1	28,92	188.217	6.399	26	ctx restrito (reload)
C3-B	ctx-2048	2	38,05	38.542	6.411	26	ctx restrito

Fonte: Autor.

Médias por Configuração

Tabela 8.2: Médias calculadas por sub-configuração (metodologia tail(2))

Config	Descrição	TTFT médio (s)	Total médio (ms)	RAM média (MB)
C1-A	padrão, query curta	59,7	60.108	5.898
C1-B	padrão, query longa	550,1	551.835	6.228

Config	Descrição	TTFT médio (s)	Total médio (ms)	RAM média (MB)
C2-A	concorrente (2 req)	111,7	173.597	5.331
C2-B	RAG pipeline ativo	69,6	70.091	6.493
C3-A	modelo 1.5b	27,4	43.433	6.609
C3-B	ctx=2048	33,5	113.380	6.405

Fonte: Autor.

Gráfico de Latência TTFT

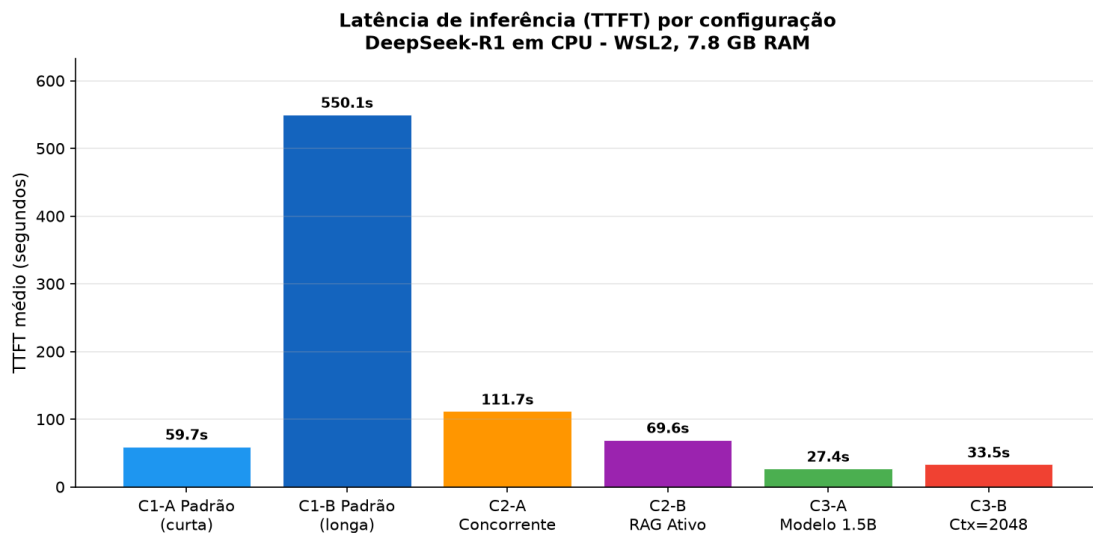


Figura 8.1: Latência de inferência (TTFT medio) por configuração - DeepSeek-R1 em CPU, WSL2

Fonte: Autor.

Gráfico de Uso de RAM

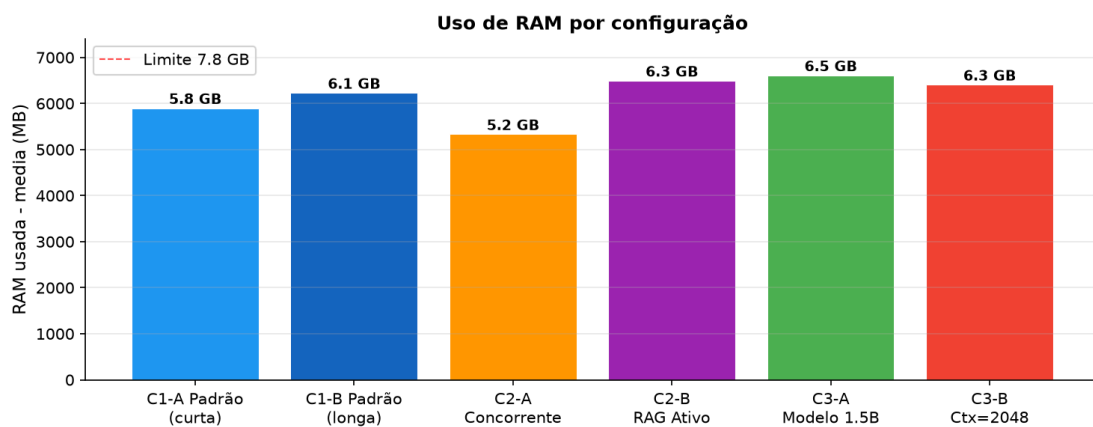


Figura 8.2: Uso de RAM médio por configuração

Fonte: Autor.

9-Análise dos Resultados

Q16 - Impacto do tamanho da query no tempo de resposta (C1-A vs C1-B)

A query longa (C1-B) levou em média 550,1 s de TTFT, contra 59,7 s da query curta (C1-A), representando uma diferença de 9,2x no tempo de resposta para a mesma configuração de hardware e

modelo. A causa direta é a quantidade de tokens gerados: o modelo precisa produzir muito mais conteúdo para uma resposta detalhada. Como a inferência em CPU tem custo linear por token, a latência cresce proporcionalmente ao tamanho da saída esperada. Isso confirma que o gargalo do sistema não está na entrada (o prompt é lido rapidamente), mas na geração token a token.

Q17 - Consistência entre repetições

A variância foi alta em todas as configurações. Em C1-A, as duas últimas repetições apresentaram 82,90 s e 36,57 s (amplitude de 46 s, ~56%). Em C1-B, 526,49 s e 573,77 s (amplitude de 47 s, ~8%). Em C2-A, 139,07 s e 84,42 s (amplitude de 55 s, ~39%). Os fatores que causam variação são: decisões do escalonador CFS do Linux sobre distribuição de time slice entre as 11 threads do llama-server; crescimento dinâmico da KV-cache durante a geração; e possível competição de memória com processos do WSL2/Windows em background.

Q18 - Comportamento de RAM ao longo dos experimentos

O uso de RAM cresceu de ~5.315 MB (C2-A, onde o modelo foi brevemente descarregado) até ~6.616 MB (C3-A, com dois modelos carregados simultaneamente). Esse crescimento tem dois componentes: a KV-cache que o Ollama mantém em memória entre requisições para evitar reprocessar tokens repetidos; e a carga cumulativa de modelos. No bloco C3-A, o deepseek-r1:1.5b foi carregado sem descarregar o 7b, elevando o uso para ~6,6 GB (4,7 GB do 7B + ~1 GB do 1.5B + overhead de runtime). Em nenhum momento o sistema atingiu swap, mas o uso se aproximou do limite de 7,8 GB disponíveis.

Q19 - Impacto da concorrência (C2-A)

No experimento C2-A, duas requisições foram enviadas simultaneamente ao Ollama. Os resultados revelam que o Ollama serializa as requisições internamente: o tempo total de wall-clock (230.013 ms e 117.180 ms) foi significativamente maior que o TTFT de cada requisição, pois a segunda aguardou na fila enquanto a primeira era processada. Isso contrasta com arquiteturas de inferência com GPU, onde batching verdadeiro permite processar múltiplas requisições em paralelo. Na CPU, o llama-server usa todos os núcleos disponíveis para uma única geração, não sobrando capacidade para paralelismo real.

Q20 - RAG pipeline ativo vs inferência direta (C2-B vs C1-A)

C2-B (RAG ativo, mesma query curta): TTFT médio de 69,6 s vs C1-A de 59,7 s. Diferença de +9,9 s (+17%). O RAM médio de C2-B foi de 6.493 MB vs 5.898 MB de C1-A, uma diferença de ~595 MB que corresponde à memória consumida pela pilha RAG (Streamlit, ChromaDB com vetores indexados, dependências LangChain). A hipótese é que a memória extra consumida pelo RAG reduziu o espaço disponível para a KV-cache do modelo, forçando mais operações de alocação e, possivelmente, aumentando a latência de acesso à memória durante a inferência. Como ambas as configurações usaram a mesma query simples sem passar pelo pipeline RAG, a lentidão vem puramente da pressão de memória.

Q21 - Impacto de modelo menor (C3-A: deepseek-r1:1.5b)

O deepseek-r1:1.5b obteve TTFT médio de 27,4 s, comparado a 59,7 s do 7B para a mesma query curta, uma redução de 54% na latência. O modelo 1.5B tem apenas ~1 GB em GGUF Q4_K_M vs 4,7 GB do 7B, o que resulta em: menos pesos para multiplicar por ativação em cada camada; modelo menor cabe em mais caches de CPU (L3), reduzindo stalls de memória; número de camadas e tamanho de atenção menores (menos operações por token). O custo foi 26 threads observadas (vs 15

do 7B), pois o segundo modelo criou um novo processo llama-server adicional que coexistiu com o do 7B.

Q22 - Impacto do tamanho de contexto (C3-B: num_ctx=2048)

C3-B usou num_ctx=2048 (vs o padrão de 8192). O TTFT médio foi de 33,5 s, similar ao C3-A com modelo 1.5B, representando redução de 44% frente ao C1-A (59,7 s). A redução de contexto de 8192 para 2048 tokens diminui a KV-cache máxima alocada em ~4x, liberando RAM para outras operações. A repetição 1 de C3-B teve total de 188.217 ms mas TTFT de apenas 28,92 s: a discrepância ocorreu porque mudar num_ctx forçou o Ollama a recarregar o modelo com nova alocação de KV-cache, adicionando ~160 s d e overhead de reload. A repetição 2, com o modelo já carregado no novo tamanho, levou apenas 38.542 ms.

Q23 - Syscall dominante no ollama serve durante inferência

Conforme documentado na Parte B, a syscall futex representou 76,59% das chamadas no processo ollama serve. O futex (Fast Userspace muTEX) coordena as 11 threads do llama-server sem precisar entrar no kernel quando não há contenção, minimizando o custo de sincronização. Cada camada do transformer requer uma barreira de sincronização entre threads, e com centenas de camadas por inferência, o futex e a syscall mais frequente de longe.

Q24 - Gargalos do sistema

Os experimentos evidenciam três gargalos distintos. O gargalo primário é a CPU: sem GPU, cada token requer multiplicações matriciais densas nas 11 threads do llama-server. O modelo 7B demora em média 0,6-1,5 s por token, resultando em latências de 27-550 s dependendo do tamanho da saída. O gargalo secundário é a memória RAM: com 7,8 GB disponíveis e o modelo 7B consumindo 4,7 GB, sobram apenas ~3 GB para o sistema, RAG e KV-cache. Quando o RAG ativo ou um segundo modelo consomem parte desse espaço, a pressão de memória aumenta a latência. O gargalo terciário é a serialização do Ollama: como demonstrado em C2-A, o servidor não processa requisições em paralelo na CPU, criando fila de espera proporcional ao número de clientes simultâneos.

Q25 - Gerenciamento de recursos pelo SO durante a inferência

Escalonamento de CPU (CFS): Distribuiu os time slices entre as 11 threads do llama-server e os demais processos do sistema. O llama-server recebeu ~97% de uso de CPU por ser CPU-bound e não realizar bloqueios desnecessários.

Gerenciamento de memória: O modelo foi mapeado via mmap no carregamento, permitindo ao kernel paginar sob demanda. A KV-cache cresce dinamicamente com malloc; em C3-B, a mudança de num_ctx forçou uma realocação do heap inteiro, visível nos ~160 s d e overhead de reload.

Sincronização de threads: O kernel atendeu futex calls para as barreiras entre camadas do transformer, com baixíssima contenção (retorno imediato em userspace na maioria das chamadas).

Escalonamento de I/O: O Ollama serve requisições HTTP via epoll_pwait no loopback, garantindo que a chegada de uma segunda requisição (C2-A) fosse detectada e enfileirada sem busy-waiting.

10-Conclusão

Os experimentos confirmam que LLMs 7B em CPU são viáveis para uso acadêmico, mas revelam limites claros: latência mínima de ~27 s (modelo 1.5B) a ~60 s (7B, query curta) e até 574 s para respostas longas. A redução de contexto (C3-B, -44% de latência) e o uso de modelos menores (C3-A, -54%) são as alavancas mais efetivas para reduzir a latência sem hardware especializado.

A relação entre SO e LLM é profunda: o escalonador CFS, o gerenciador de memória virtual, o futex de sincronização e o epoll de I/O são os pilares que tornam possível rodar um modelo de 4,7 GB em um laptop com 7,8 GB de RAM. Os experimentos tornaram visíveis mecanismos que normalmente são transparentes ao usuário final, demonstrando como o SO é o substrato indispensável de qualquer aplicação de IA local.

A variância observada entre repetições (até 56% em C1-A) ressalta que medições únicas são insuficientes para caracterizar o desempenho de sistemas LLM em CPU: é necessário múltiplas repetições e análise estatística robusta para conclusões confiáveis.

11-Declaração de Uso de Inteligência Artificial

Declaro que utilizei ferramentas de inteligência artificial (Claude, da Anthropic) como auxílio na elaboração deste relatório, nas seguintes capacidades: (1) revisão gramatical e de clareza do texto em português; (2) sugestão de estrutura para as respostas às questões Q16-Q26; (3) auxílio na depuração dos scripts Python de coleta e visualização de dados. Todos os experimentos, medições, análises e conclusões apresentados neste relatório são de minha autoria, baseados em dados reais coletados no ambiente descrito. As ferramentas de IA foram utilizadas como apoio à escrita, não como substituto do raciocínio crítico ou da execução dos experimentos.

Link do repositório: https://github.com/alexismelo/SO_UFS_2026_2_Melo_Alex

Link do vídeo: <https://youtu.be/tex8QwoLRIE>